

Interação com APIs

02

2026

SENAI - CRTI

Arquitetura REST e Verbos HTTP

Como aplicações manipulam recursos em APIs



{ REST }

O Desafio: Task Manager Corporativo

Contexto do Mundo Real

Uma equipe de desenvolvimento precisa criar o **front-end de um sistema de gerenciamento de tarefas corporativas**. A API já está disponível e expõe endpoints para todas as operações essenciais do negócio.

Sua missão: entender como cada ação da interface se traduz em uma requisição HTTP adequada ao servidor.

Operações disponíveis na API

- Listar todas as tarefas
- Cadastrar nova tarefa
- Editar tarefa existente
- Marcar tarefa como concluída
- Excluir tarefa

❓ **Pergunta-problema:** Como o front-end informa ao servidor exatamente o que deseja fazer com um recurso?

APIs no Mundo do Trabalho

Instagram

Carrega posts do feed, comentários e curtidas ao rolar a tela

App Bancário

Cadastra chave Pix, realiza transferências e consulta extrato

E-commerce

Atualiza itens no carrinho, finaliza pedido e rastreia entrega

Sistema Escolar

Remove matrícula, altera turma e registra frequência

ERP Corporativo

Altera cadastro de fornecedor, atualiza estoque e gera relatórios

 **Mensagem-chave:** Toda ação da interface normalmente corresponde a uma operação HTTP específica executada contra a API.

O que é REST?

REST (Representational State Transfer) é um **estilo arquitetural para comunicação entre sistemas baseado em recursos**.

Proposto por Roy Fielding em 2000, tornou-se o padrão dominante para construção de APIs Web modernas.

Recursos

Qualquer entidade do sistema identificável por uma URL única

Endpoints

URLs que representam recursos e aceitam operações HTTP

Stateless

Cada requisição é independente e contém todas as informações necessárias

Exemplos de recursos em APIs REST:

```
/usuarios  
/produtos  
/tarefas  
/pedidos
```

- ❏ Cada recurso é identificado por uma URL clara e padronizada. O **protocolo HTTP** define como o cliente interage com esses recursos — usando métodos (verbos) que indicam a intenção da operação.

Operações CRUD e os Verbos HTTP

REST conversa de forma natural com o modelo CRUD — as quatro operações fundamentais de persistência de dados. Cada operação CRUD possui um verbo HTTP correspondente.

Operação CRUD	Verbo HTTP	Ação	Exemplo
Create	POST	Criar novo recurso	POST /tarefas
Read	GET	Consultar recurso	GET /tarefas
Update	PUT / PATCH	Atualizar recurso	PUT /tarefas/10
Delete	DELETE	Remover recurso	DELETE /tarefas/10

- ✔ Compreender essa correspondência é o primeiro passo para consumir qualquer API REST com Angular ou qualquer outro framework front-end.

GET — Consultar Recursos

O método **GET** é utilizado para **consultar recursos** no servidor. É o verbo mais utilizado na web — toda vez que você acessa uma página ou carrega uma lista de dados, um GET está sendo realizado.

Exemplos de uso:

```
GET /tarefas
GET /tarefas/10
GET /usuarios
```



Listar

Retorna coleção de recursos



Buscar

Retorna um recurso pelo ID



Detalhar

Consulta propriedades completas



Operação segura: GET nunca altera dados no servidor. É idempotente — múltiplas chamadas produzem o mesmo resultado.

HTTP

HTTP GET Request

POST — Criar Recursos

Definição

O método **POST** é utilizado para **criar um novo recurso** no servidor. Os dados do novo recurso são enviados no **corpo (body)** da requisição no formato JSON.

Endpoint:

POST /tarefas

Características:

- Envia dados no body da requisição
- O servidor cria o recurso e retorna o ID gerado
- Resposta esperada: **201 Created**
- Não é idempotente — chamadas repetidas criam múltiplos registros

Exemplo de payload (body):

```
{  
  "titulo": "Estudar Angular",  
  "status": "pendente"  
}
```

i O servidor recebe o JSON, valida os dados, persiste no banco de dados e retorna o recurso criado com o ID atribuído automaticamente.

Fluxo da operação:

Cliente envia JSON

Servidor valida

Retorna 201 com ID

PUT vs. PATCH — Atualizar Recursos

PUT — Substituição Completa

O método **PUT substitui o recurso inteiro**. Todos os campos devem ser enviados no body, mesmo os que não foram alterados.

```
PUT /tarefas/10
```

```
{  
  "titulo": "Estudar Angular",  
  "status": "concluida"  
}
```

⚠ Se omitir um campo no PUT, o servidor pode sobrescrever com valor nulo ou padrão. Envie sempre o recurso completo.

PUT substitui tudo

Recurso inteiro obrigatório no body

PATCH — Alteração Parcial

O método **PATCH altera parcialmente o recurso**. Apenas os campos que precisam ser modificados são enviados no body.

```
PATCH /tarefas/10
```

```
{  
  "status": "concluida"  
}
```

✅ PATCH é mais eficiente para atualizações pontuais — como marcar uma tarefa como concluída sem reenviar todos os outros campos.

PATCH altera parte

Somente os campos modificados

DELETE — Remover Recursos

Uso do método DELETE:

```
DELETE /tarefas/10
```

O método **DELETE** solicita ao servidor a **remoção de um recurso específico**, identificado pelo seu ID na URL. Geralmente não possui body na requisição.

Resposta esperada:

- **200 OK** — com body de confirmação
- **204 No Content** — remoção sem retorno de dados
- **404 Not Found** — recurso não existe

Remoção Física

O registro é **deletado permanentemente** do banco de dados. Operação irreversível — use com cautela.

Remoção Lógica

O registro recebe um campo como **ativo: false** ou **deletedAt**. Preserva histórico e integridade referencial.

⚠ Cuidado com integridade referencial: deletar um recurso que possui dependências pode causar erros em cascata. APIs bem projetadas retornam erro 409 Conflict nesses casos.

Códigos de Status HTTP

Status code é a resposta do servidor sobre o resultado da operação. Todo retorno HTTP inclui um código numérico de 3 dígitos que indica se a requisição foi bem-sucedida ou o tipo de erro ocorrido.

2xx — Sucesso

- **200 OK** — operação bem-sucedida
- **201 Created** — recurso criado
- **204 No Content** — sucesso sem dados

4xx — Erro do Cliente

- **400 Bad Request** — dados inválidos
- **401 Unauthorized** — não autenticado
- **403 Forbidden** — acesso negado
- **404 Not Found** — não encontrado

5xx — Erro do Servidor

- **500 Internal Server Error** — falha interna

Interpretando Status Codes na Prática

Código	Nome	Quando ocorre	Verbo típico
200	OK	Consulta ou atualização bem-sucedida	GET, PUT, PATCH
201	Created	Recurso criado com sucesso	POST
204	No Content	Operação OK, sem corpo de resposta	DELETE, PUT
400	Bad Request	Dados enviados são inválidos ou malformados	POST, PUT, PATCH
401	Unauthorized	Token ausente ou expirado	Todos
403	Forbidden	Autenticado, mas sem permissão	Todos
404	Not Found	Recurso não existe ou ID inválido	GET, DELETE, PATCH
500	Internal Error	Falha inesperada no servidor	Todos

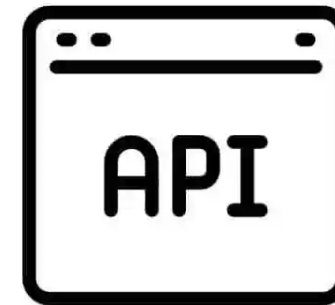


No Angular, o `HttpClient` trata automaticamente os status 4xx e 5xx como erros, permitindo que você os capture no bloco `catchError` do RxJS.

DEMONSTRAÇÃO PRÁTICA

JSONPlaceholder — API de Teste

O **JSONPlaceholder** é uma API REST pública e gratuita, ideal para praticar requisições HTTP sem precisar de back-end próprio. Vamos explorar seus endpoints de posts.



JSON Placeholder

Testando na Prática com JSONPlaceholder

GET — Listar todos os posts

```
GET https://jsonplaceholder.typicode.com/posts
```

Retorna um array JSON com 100 posts. Status esperado: **200 OK**.

GET — Buscar post por ID

```
GET https://jsonplaceholder.typicode.com/posts/1
```

Retorna um único objeto JSON com os dados do post de ID 1.
Status esperado: **200 OK**.

POST — Criar novo post

```
POST https://jsonplaceholder.typicode.com/posts
```

Envia um body JSON no corpo da requisição. O JSONPlaceholder simula a criação e retorna o objeto com ID atribuído. Status esperado: **201 Created**.

📌 Nesta API de teste, os dados não são persistidos de verdade — mas o comportamento dos status codes e das respostas é idêntico ao de uma API real. Perfeito para aprender!

Task Manager × Verbos HTTP

Voltando ao nosso cenário de aprendizagem: veja como cada funcionalidade do **Task Manager corporativo** se mapeia a um verbo HTTP e a um status code esperado.

Ação na Interface	Verbo HTTP	Endpoint	Status Esperado
Exibir lista de tarefas	GET	/tarefas	200 OK
Abrir detalhe de tarefa	GET	/tarefas/10	200 OK
Cadastrar nova tarefa	POST	/tarefas	201 Created
Editar todos os campos	PUT	/tarefas/10	200 OK
Marcar como concluída	PATCH	/tarefas/10	200 OK
Excluir tarefa	DELETE	/tarefas/10	204 No Content

Vamos Pensar Juntos?

Debates como estes são fundamentais para o desenvolvimento do raciocínio técnico profissional. Não existe resposta única — depende do contexto e das decisões de arquitetura.

1

Atualizar cadastro inteiro ou só o campo alterado?
Qual a diferença de custo entre PUT e PATCH? Quando cada um é mais adequado para a UX e para a performance da API?

2

Deletar fisicamente ou apenas inativar?
Como a estratégia de remoção lógica impacta o histórico de dados, relatórios e auditoria no sistema corporativo?

3

Um GET pode alterar dados no banco?
Tecnicamente é possível. Mas viola princípios REST. Por que isso é considerado uma má prática arquitetural?

4

Como a API comunica sucesso ou falha?
O status code é suficiente? Quando faz sentido incluir mensagens de erro detalhadas no body da resposta?

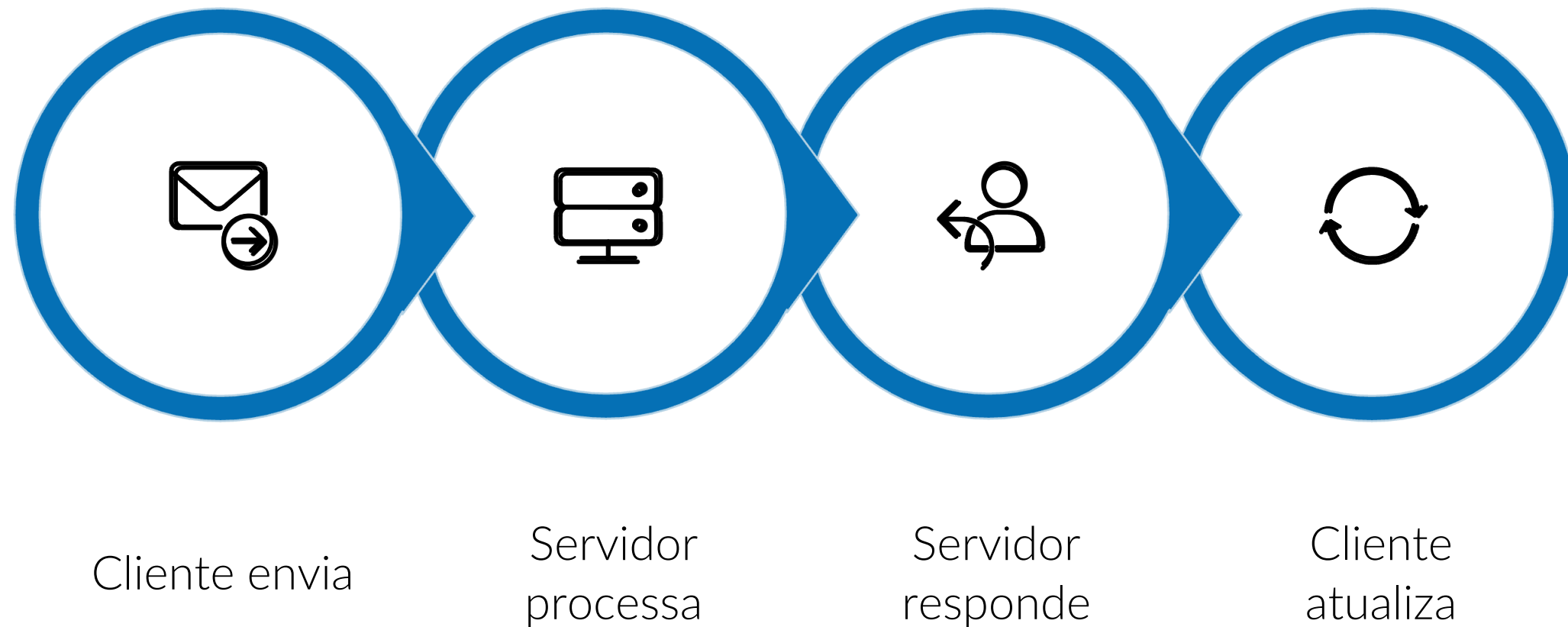
Identificando o Verbo Correto

Para cada cenário abaixo, determine o **verbo HTTP mais adequado**, o endpoint provável e o **status code esperado** em caso de sucesso. Discuta suas respostas com o grupo.

Cenário	Verbo HTTP	Endpoint (sugestão)	Status
Cadastrar novo aluno no sistema	POST	/alunos	201
Listar todos os cursos disponíveis	GET	/cursos	200
Alterar somente a senha do usuário	PATCH	/usuarios/5	200
Mudar o status de um pedido	PATCH	/pedidos/12	200
Excluir um comentário	DELETE	/comentarios/8	204

❓ Após definir os verbos, identifique também: qual status seria retornado se o recurso **não fosse encontrado**? E se o usuário **não tiver permissão**?

Fluxo Completo de uma Requisição REST



Este é o ciclo fundamental que sustenta toda comunicação entre front-end e back-end via API REST. Compreendê-lo em profundidade é a base para trabalhar com **Angular HttpClient**, interceptors, tratamento de erros e UX responsivo.

REST na Prática com Angular HttpClient

Cada verbo HTTP tem seu método no HttpClient:

GET → `this.http.get(url)`

Consulta recursos e retorna Observable com os dados

POST → `this.http.post(url, body)`

Cria recurso enviando objeto JSON no body

PUT → `this.http.put(url, body)`

Substitui recurso completo

PATCH → `this.http.patch(url, body)`

Atualiza campos específicos do recurso

DELETE → `this.http.delete(url)`

Remove o recurso identificado pela URL

Fundamentos necessários (já vistos hoje):

- Verbos HTTP e suas finalidades
- Mapeamento CRUD → HTTP
- Interpretação de status codes
- Estrutura de recursos REST

O que você aprendeu hoje

Ao concluir esta aula, você demonstra as seguintes competências previstas na MSEP:



Identificar Arquitetura REST

Reconhecer os princípios REST: recursos, endpoints, stateless e protocolo HTTP como base da comunicação.



Relacionar CRUD e HTTP

Mapear operações Create, Read, Update e Delete aos verbos HTTP correspondentes com precisão.



Diferenciar GET, POST, PUT, PATCH e DELETE

Aplicar o verbo correto para cada tipo de operação em cenários reais de desenvolvimento.



Interpretar Status Codes

Compreender famílias 2xx, 4xx e 5xx e identificar o código esperado para cada operação HTTP.

Próximos Passos

Para consolidar o aprendizado:

01

Revisite a tabela CRUD × HTTP

Pratique identificar o verbo correto para diferentes cenários do mundo do trabalho

02

Explore o JSONPlaceholder

Acesse

`jsonplaceholder.typicode.com` e teste os endpoints no navegador ou no Postman/Insomnia

✅ **Lembre-se:** Toda ação da interface corresponde a uma operação HTTP. Dominar os verbos é dominar a comunicação entre sistemas.



